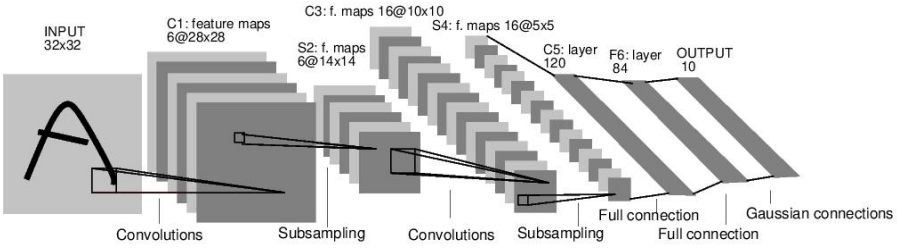
1. 问题概述

1.1 问题描述与分析

本次课设围绕深度学习展开。其中包含基于 MNIST 数据集的手写数字分类和基于给定数据集的垃圾分类问题。两个问题均在 MindSpore 框架下解决。两个问题的具体描述与分析如下：

问题 1 中，需要使用 MindSpore 在 MNIST 数据集上开发和训练一个 LeNet5 模型。MNIST 数据集的图像像素为 28*28，图像存储在 Tensor 对象中作为输入。输出是一个 10 维向量，分别对应该输入图像属于 10 个数字（0-9）的概率。根据已有的资料得知，LeNet5 模型的输入图像像素为 32*32，而 MNIST 数据集的图像像素为 28*28，因此在对数据的预处理阶段需要先对图像尺寸进行处理。其中 MindSpore 中的 MindSpore Vision 套件提供了用于下载并处理 MNIST 数据集的 Mnist 模块，可利用该函数将数据集下载、解压到指定位置并进行数据处理。



LeNet5 模型的网络结构

问题 2 中，在 CPU 环境下，使用在 OpenImage 数据集上预训练的 MobileNetV2 模型，通过 MindSpore 在垃圾分类数据集上微调(Fine-Tune)与验证。

1.2 解决问题的思路和想法

1.2.1 问题 1 的解决思路

问题 1 利用深度学习方法，设计并训练 LeNet-5 模型，对 MNIST 数据集中的手写数字图片进行分类。MNIST 数据集包含 10 类（数字 0-9），每张图片为 28×28 像素的灰度图。

在数据预处理阶段，利用 MindSpore Vision 工具包自动下载并处理 MNIST 数据（**事实**

上，如今 MNIST 数据集官网已经无法访问，即无法利用工具包从官网自动下载 MNIST 数据集，需要手动获得 MNIST 数据集，从本地读取数据集进行处理。MNIST 数据集获取途径在“参考文献”部分列出），将图片统一 resize 为 32×32 像素，以满足 LeNet-5 模型的输入需求。然后，按照 LeNet5 的结构，构建包含卷积层、池化层和全连接层的神经网络，并设置输出层节点数为 10，分别对应每个数字类别。

在模型训练过程中，采用 softmax 交叉熵损失函数作为模型优化目标。通过反向传播算法，根据训练集图片及其标签，不断调整模型参数，使模型输出的预测概率分布逐步逼近真实标签的分布。训练过程中，定期保存模型参数，便于后续微调或推理。

模型训练完成后，在测试集上进行评估，计算模型的分类准确率，用于衡量模型的泛化能力。最终，通过加载训练好的模型参数，对新的手写数字图片进行预测，并输出模型识别结果。

1.2.2 问题 2 的解决思路

问题 2 主要是借助 MindSpore 在基于 OpenImage 数据集上预训练的 MobileNetV2 模型上进行微调与验证，已有的垃圾分类数据集共有 26 类，每张图片为 224*224 的图片。

在数据预处理阶段，由于 MobileNetV2 的代码默认使用 ImageFolder 格式管理数据集，因此需要先把每一类图片整理成一个单独的文件夹（已有的数据集已经按照该种结构组织了）。接着从 MindSpore 的开源仓库中下载 MobileNetV2 的预训练模型和相关的脚本文件。使用预训练模型进行微调的话，根据是否冻结一部分网络又分为不冻结 MobileNetV2Backbone 部分和冻结 MobileNetV2Backbone 部分两种方式。在网络中包含 MobileNetV2Backbone 和 MobileNetV2Head 两部分，如果选择冻结 MobileNetV2Backbone 部分，即在模型训练中不需要训练得到 MobileNetV2Backbone 的参数，MobileNetV2Backbone 网络参数从已经训练好的 ckpt 模型中得到（参数 pretrain_ckpt 非空）。由于电脑 CPU 运行速度慢，所以仅支持 fine_tune 方式下的冻结 MobileNetV2Backbone 部分。即：参数 freeze_layer 为"backbone"，参数 pretrain_ckpt 非空。**只在垃圾分类数据集上训练 MobileNetV2Head 部分。**

模型训练过程中，定义 MobileNetV2Backbone 网络，加载 mobilenetv2_cpu_gpu.ckpt 到 Backbone 中，并利用 Backbone 将数据集特征化，生成特征数据。定义 MobileNetV2Head 网络，以特征数据作为输入，训练 MobileNetV2Head 网络。

模型训练完成后，在验证集上验证模型的性能，通过 acc 计算模型分类的准确率。

2. 算法设计

2.1 问题 1 手写数字识别

算法原理：

LeNet5 是一种卷积神经网络（CNN），其核心思想是通过多层卷积、池化和全连接操作，自动提取图像特征并进行分类预测。其中卷积层用来提取局部特征，池化层降低特征维度，全连接层完成抽象和最终分类。在训练过程中，LeNet5 通过反向传播算法不断调整网络参数，使模型输出逐渐逼近真实标签。

算法功能：

本算法实现了基于 MindSpore 框架下的 LeNet5 模型，能够自动识别输入的图像，将其分为 0-9 数字中的一类。

设计思路：

1.输入层：将每张手写数字图片处理为 32×32 灰度图像作为模型的输入。（借助 mindvision.dataset 中提供的 Mnist 函数 resize 即可）

（nn. 指 MindSpore 框架的神经网络（Neural Network）模块）

2.卷积层与池化层：使用 nn.Conv2d 实现卷积操作，nn.MaxPool2 实现池化操作，用于特征提取和下采样。

3.激活函数：在每个卷积层和全连接层后，通常使用 nn.ReLU 作为激活函数，使网络具有非线性表达能力，提升模型性能。

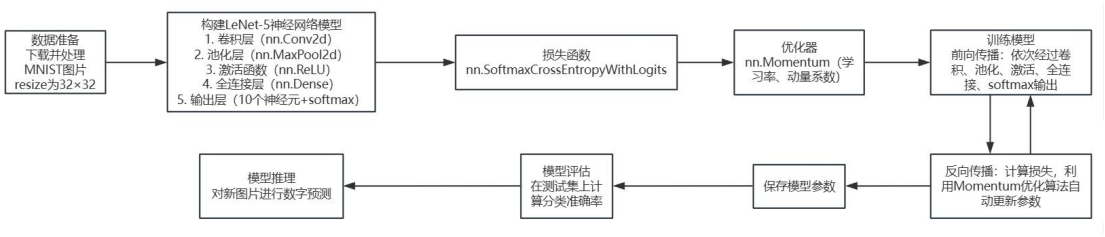
4.全连接层：使用 nn.Dense 实现全连接操作，将高维特征映射到输出空间。

5.输出层：最后一层全连接输出 10 个神经元，表示数字 0-9 的分类得分，后接 softmax 函数用于概率归一化。

6.损失函数：使用 nn.SoftmaxCrossEntropyWithLogits 作为损失函数，对模型输出概率与真实标签进行比较，指导模型优化。

7.优化器：使用 Momentum 这一优化算法（一种加速梯度下降收敛的优化算法），通过前向传播和反向传播计算损失并更新参数，迭代优化模型性能。

算法流程图：



2.2 问题 2 垃圾分类

算法原理：

MobileNetV2 是一种轻量级深度神经网络，其主要特点是采用深度可分离卷积（Depthwise Separable Convolution）替换了普通卷积。MobileNetV2 在 MobileNetV1 的基础上引入了线性瓶颈（Linear Bottleneck)和倒残差（Inverted Residual)来提高网络的表征能力。

算法功能：

本算法实现了基于 MindSpore 框架下的 MobileNetV2 微调模型，能够自动识别输入的图像，对其进行垃圾分类。

设计思路：

1.输入层：将每张 224*224 的垃圾 RGB 图像输入。（由于彩色的垃圾图片就是 3 通道，所以不需要在原有参数上修改通道数）。

（nn. 指 MindSpore 框架的神经网络（Neural Network）模块）

2.初始卷积层：使用一个标准卷积（nn.Conv2d）和 BatchNorm（nn.BatchNorm2d），结合 ReLU6 激活（nn.ReLU6），对输入进行初步特征提取和下采样。

3.倒残差结构（Inverted Residual Block）：这是 MobileNetV2 的核心创新。每个 block 主要包括：

a）1×1 逐点卷积升维（通道扩展）；

b）3×3 深度可分离卷积（Depthwise Conv）进行空间特征提取；

c）1×1 逐点卷积降维（通道还原）；

d）每步均配有 BatchNorm 和 ReLU6 激活；

e）若满足条件（步长为 1 且输入输出通道数一致），采用残差连接，将输入直接加到输出上，实现特征复用和更好的梯度传递。

通过设置不同的扩展倍数、输出通道和步长，堆叠多个 block，实现高效的特征提取和

下采样。

4. 最终卷积层：使用 1×1 卷积将特征通道提升至较高维度（如 1280），为分类头部提供丰富的特征表达。

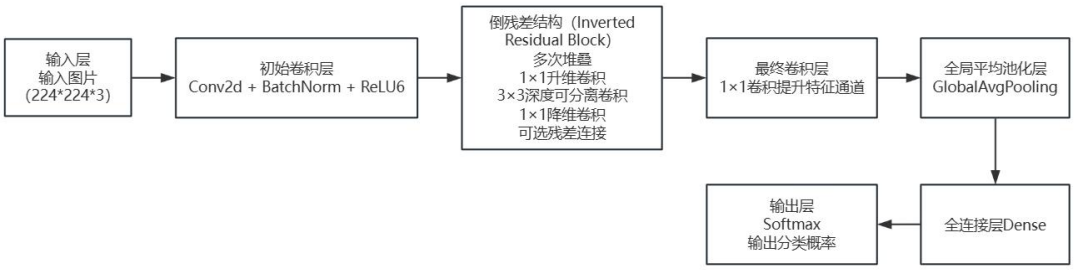
5. 全连接层：使用 nn.Dense 实现全连接操作，将高维特征映射到输出空间。

6. 输出层：最后一层接 softmax 函数用于概率归一化。

7. 损失函数：使用 nn.SoftmaxCrossEntropyWithLogits 作为损失函数，对模型输出概率与真实标签进行比较，指导模型优化。

8. 优化器：使用 Momentum 这一优化算法（一种加速梯度下降收敛的优化算法），通过前向传播和反向传播计算损失并更新参数，迭代优化模型性能。（除此之外也支持多种优化算法的选择，如 Adam 等）。

算法流程图：



3. 算法实现

3.1 问题 1 手写数字识别

实现细节：

本算法基于 MindSpore 深度学习框架，结合 MindVision 提供的 LeNet 网络模型接口，搭建 LeNet5 卷积神经网络对 MNIST 手写数字图片进行分类识别。主要实现步骤包括数据集加载与预处理、模型构建、模型训练与保存、模型评估以及推理预测等流程。具体涉及的核心函数及其输入输出如下所述。

核心函数及其输入输出：

1. **Mnist 类**：用于加载 MNIST 数据集并进行处理

```
download_train = Mnist(path="./mnist", split="train", batch_size=32, repeat_num=1,
shuffle=True, resize=32, download=False)

download_eval = Mnist(path="./mnist", split="test", batch_size=32, resize=32,
download=False)
```

输入：

path：数据集存储路径。

split：数据集的划分，"train" 表示训练集，"test" 表示测试集。

batch_size：每个批次的数据量。

repeat_num：数据集重复的次数。

shuffle：是否打乱数据。

resize：将图像调整到指定大小。

download：是否下载数据集（False 表示使用本地数据集）。

输出：

dataset_train 和 dataset_eval：分别为训练集和测试集的迭代器。

2. **lenet 函数：**用于接入 lenet 模型

```
network = lenet(num_classes=10, pretrained=False)
```

输入：

num_classes：分类的类别数（MNIST 数据集有 10 个类别，对应数字 0-9）。

pretrained：是否使用预训练模型（这里设置为 False，表示不使用预训练模型）。

输出：

network：LeNet 网络模型。

3. **nn.SoftmaxCrossEntropyWithLogits 函数：**该模型使用的损失函数

```
net_loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
```

输入：

sparse：是否使用稀疏标签（MNIST 数据集的标签是稀疏的）（标签是稀疏的，是因为每个标签只是一个简单的整数，而不是一个完整的向量）。

reduction：损失值的归约方式（'mean' 表示取平均值）。

4. **nn.Momentum 函数：**该模型使用的优化器，加速梯度下降，提高收敛速度

```
net_opt = nn.Momentum(network.trainable_params(), learning_rate=0.01, momentum=0.9)
```

输入：

network.trainable_params()：网络中可训练的参数。

learning_rate：学习率，此处学习率为 0.01。

momentum：动量因子，此处为 0.9。

5. **model.train 函数：**用于模型训练

```
model.train(10, dataset_train, callbacks=[ckpoint, LossMonitor(0.01, 1875)])
```

输入：

Epoch：训练的轮数，在这里是 10。

dataset_train：训练数据集。

callbacks：训练过程中的回调函数（保存模型和监控损失值）。

6. **model.eval 函数：**用于在测试集上进行模型评估

```
acc = model.eval(dataset_eval)
print("{}".format(acc))
```

输入：

dataset_eval：测试数据集。

输出：

acc：模型的准确率。

数据结构定义：

1. **字典：**

- 每个 batch 都是一个字典，包含灰度图片张量数据和每个照片的数字标签张量。
- 保存模型权重的.ckpt 文件内部为网络各层的参数字典。

2. **Numpy 数组：**

- 输出的预测类别是 numpy 数组的形式，如 Predicted: "[7 2 1 0 4 1]", Actual: "[7 2 1 0 4 1]"。

3.2 问题 2 垃圾分类

实现细节：

本算法基于 MindSpore 深度学习框架，搭建 MobileNetV2 轻量级卷积神经网络，实现对垃圾图片的高效分类。主要实现步骤包括网络结构的模块化定义、特征提取主干（Backbone）和分类头部（Head）的组装、模型训练与评估等。其中在垃圾分类数据集上只对分类头部 Head 进行训练，Backbone 直接读取预训练模型中的参数。具体涉及的模块输入输出以及核心函数及其输入输出如下所述。

整体网络的输入输出：

1. Backbone ：

输入：形如(batch_size, 3, H, W)的图片张量（如 224×224 RGB 图像）。

输出：经过特征提取和通道提升后的高维特征张量。

2. Head ：

输入：主干输出的特征张量。

输出：shape 为(batch_size, num_classes)的分类概率。

3. 整体 MobileNetV2 ：

输入：图片张量。

输出：各类别的概率。

核心函数及其输入输出：

1.ConvBNReLU 类：卷积+归一化+激活的基础单元

```
class ConvBNReLU(nn.Cell):
    def __init__(self, in_planes, out_planes, kernel_size=3, stride=1, groups=1):
        super(ConvBNReLU, self).__init__()
        padding = (kernel_size - 1) // 2
        in_channels = in_planes
        out_channels = out_planes
        if groups == 1:
            conv = nn.Conv2d(in_channels, out_channels, kernel_size, stride, pad_mode='pad', padding=padding)
        else:
            out_channels = in_planes
            conv = nn.Conv2d(in_channels, out_channels, kernel_size, stride, pad_mode='pad',
                             padding=padding, group=in_channels)

        layers = [conv, nn.BatchNorm2d(out_planes), nn.ReLU6()]
        self.features = nn.SequentialCell(layers)

    def construct(self, x):
        output = self.features(x)
        return output
```

输入：

in_planes：输入通道数

out_planes：输出通道数

kernel_size：卷积核大小

stride：步幅

groups：分组卷积数

输出：

out：经过卷积、BN、ReLU6 激活后的特征

2.GlobalAvgPooling 类：全局平均池化层

```
class GlobalAvgPooling(nn.Cell):

    def __init__(self):
        super(GlobalAvgPooling, self).__init__()
```

```python
        self.mean = P.ReduceMean(keep_dims=False)

    def construct(self, x):
        x = self.mean(x, (2, 3))
        return x
```

输入：

x：输入特征张量，shape 为(batch_size, channels, H, W)

输出：

out：池化后的特征张量，shape 为(batch_size, channels)

3. InvertedResidual 类：倒残差结构单元

```python
class InvertedResidual(nn.Cell):
    def __init__(self, inp, oup, stride, expand_ratio):
        super(InvertedResidual, self).__init__()
        assert stride in [1, 2]

        hidden_dim = int(round(inp * expand_ratio))
        self.use_res_connect = stride == 1 and inp == oup

        layers = []
        if expand_ratio != 1:
            layers.append(ConvBNReLU(inp, hidden_dim, kernel_size=1))
        layers.extend([
            # dw
            ConvBNReLU(hidden_dim, hidden_dim,
                       stride=stride, groups=hidden_dim),
            # pw-linear
            nn.Conv2d(hidden_dim, oup, kernel_size=1,
                      stride=1, has_bias=False),
            nn.BatchNorm2d(oup),
        ])
        self.conv = nn.SequentialCell(layers)
        self.add = TensorAdd()
        self.cast = P.Cast()

    def construct(self, x):
        identity = x
        x = self.conv(x)
        if self.use_res_connect:
            return self.add(identity, x)
        return x
```

输入：

inp：输入通道数

oup：输出通道数

stride：步幅

expand_ratio：通道扩展比例

输出：

out：残差结构输出

4. MobileNetV2Backbone 类：特征提取主干

```
class MobileNetV2Backbone(nn.Cell):
    def __init__(self, width_mult=1., inverted_residual_setting=None, round_nearest=8,
                 input_channel=32, last_channel=1280):
        super(MobileNetV2Backbone, self).__init__()
        block = InvertedResidual
        # setting of inverted residual blocks
        self.cfgs = inverted_residual_setting
        if inverted_residual_setting is None:
            self.cfgs = [
                # t, c, n, s
                [1, 16, 1, 1],
                [6, 24, 2, 2],
                [6, 32, 3, 2],
                [6, 64, 4, 2],
                [6, 96, 3, 1],
                [6, 160, 3, 2],
                [6, 320, 1, 1],
            ]

        # building first layer
        input_channel = _make_divisible(input_channel * width_mult, round_nearest)
        self.out_channels = _make_divisible(last_channel * max(1.0, width_mult), round_nearest)
        features = [ConvBNReLU(3, input_channel, stride=2)]
        # building inverted residual blocks
        for t, c, n, s in self.cfgs:
            output_channel = _make_divisible(c * width_mult, round_nearest)
            for i in range(n):
                stride = s if i == 0 else 1
                features.append(block(input_channel, output_channel, stride, expand_ratio=t))
                input_channel = output_channel
        # building last several layers
        features.append(ConvBNReLU(input_channel, self.out_channels, kernel_size=1))
        # make it nn.CellList
        self.features = nn.SequentialCell(features)
```

输入：

x：输入图片张量，shape 为(batch_size, 3, H, W)

输出：

x：堆叠倒残差结构后得到的深层特征张量

5. MobileNetV2Head 类：分类头部

```
class MobileNetV2Head(nn.Cell):
    def __init__(self, input_channel=1280, num_classes=1000, has_dropout=False, activation="None"):
        super(MobileNetV2Head, self).__init__()
        # mobilenet head
        head = ([GlobalAvgPooling(), nn.Dense(input_channel, num_classes, has_bias=True)] if not has_dropout else
                [GlobalAvgPooling(), nn.Dropout(0.2), nn.Dense(input_channel, num_classes, has_bias=True)])
        self.head = nn.SequentialCell(head)
        self.need_activation = True
        if activation == "Sigmoid":
            self.activation = P.Sigmoid()
        elif activation == "Softmax":
```

```
        self.activation = P.Softmax()
    else:
        self.need_activation = False
    self._initialize_weights()
```

输入：

x：主干输出的特征张量

输出：

out：分类概率（shape 为(batch_size, num_classes)）

6. **MobileNetV2 类：**整体网络结构

```
class MobileNetV2(nn.Cell):
    def __init__(self, num_classes=1000, width_mult=1., has_dropout=False, inverted_residual_setting=None, \
        round_nearest=8, input_channel=32, last_channel=1280):
        super(MobileNetV2, self).__init__()
        self.backbone = MobileNetV2Backbone(width_mult=width_mult, \
            inverted_residual_setting=inverted_residual_setting, \
            round_nearest=round_nearest, input_channel=input_channel, last_channel=last_channel).get_features
        self.head = MobileNetV2Head(input_channel=self.backbone.out_channel, num_classes=num_classes, \
            has_dropout=has_dropout).get_head

    def construct(self, x):
        x = self.backbone(x)
        x = self.head(x)
        return x
```

输入：

x：输入图片张量

输出：

out：每个类别的得分或概率

7. **损失函数、优化器、模型训练与评估的函数**参考“手写数字识别”中提到的函数。在此不再赘述。

数据结构定义：

1. **字典与张量（Tensor）：**

- 每个 batch 都是一个字典，包含灰度图片张量数据和每个照片的数字标签张量。
- 保存模型权重的.ckpt 文件内部为网络各层的参数字典。

2. **Numpy 数组：**

- 推理输出的预测类别通常被转为 Numpy 数组，便于与实际标签对比和展示

4. 实验结果

4.1 问题 1 手写数字识别

本问题需要实现对 MNIST 数据集上手写数字图片进行 0-9 的分类。模型一共训练了 10 个 epoch，每个 epoch 训练完会输出当前 epoch 训练的相关信息。

Epoch: [x / y] ：x 表示当前完成的 epoch 数，y 表示总共需要完成的 epoch 数。

step: [z / w]：z 表示当前 epoch 中完成的 step 数，w 表示当前 epoch 中总共需要完成的 step 数。

loss: [x / y] ：x 表示当前 step 的损失值，y 表示在当前 epoch 中所有 step 的平均损失。

lr：表示当前使用的 learning rate（学习率）。本模型的学习率设置为固定的（0.01），但学习率可以随着训练过程动态调整。

Epoch time：表示完成一个 epoch 所需的总时间。一个 epoch 指的是模型在整个训练数据集上完成一次前向和反向传播的过程。

per step time：表示每个 step 的平均处理时间。一个 step 通常指的是模型处理一个批次（batch）的数据所需的时间。

avg loss：是平均损失。

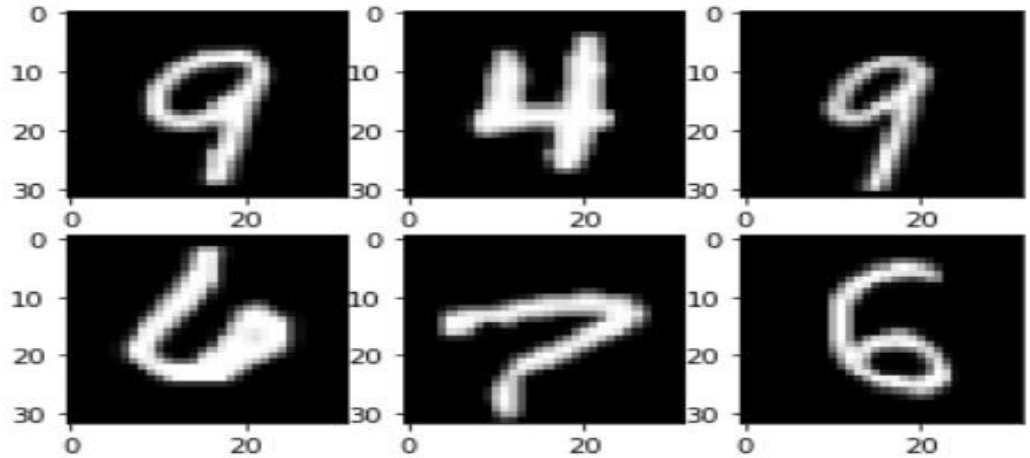
```
Epoch:[  0/ 10], step:[ 1875/ 1875], loss:[0.109/1.585], time:108.669 ms, lr:0.01000
Epoch time: 110376.740 ms, per step time: 58.868 ms, avg loss: 1.585
Epoch:[  1/ 10], step:[ 1875/ 1875], loss:[0.075/0.089], time:111.634 ms, lr:0.01000
Epoch time: 116500.916 ms, per step time: 62.134 ms, avg loss: 0.089
Epoch:[  2/ 10], step:[ 1875/ 1875], loss:[0.148/0.054], time:19.933 ms, lr:0.01000
Epoch time: 112802.965 ms, per step time: 60.162 ms, avg loss: 0.054
Epoch:[  3/ 10], step:[ 1875/ 1875], loss:[0.133/0.040], time:112.503 ms, lr:0.01000
Epoch time: 113419.797 ms, per step time: 60.491 ms, avg loss: 0.040
Epoch:[  4/ 10], step:[ 1875/ 1875], loss:[0.001/0.033], time:108.736 ms, lr:0.01000
Epoch time: 114399.160 ms, per step time: 61.013 ms, avg loss: 0.033
Epoch:[  5/ 10], step:[ 1875/ 1875], loss:[0.003/0.027], time:19.901 ms, lr:0.01000
Epoch time: 111621.055 ms, per step time: 59.531 ms, avg loss: 0.027
Epoch:[  6/ 10], step:[ 1875/ 1875], loss:[0.000/0.022], time:109.736 ms, lr:0.01000
Epoch time: 111215.714 ms, per step time: 59.315 ms, avg loss: 0.022
Epoch:[  7/ 10], step:[ 1875/ 1875], loss:[0.005/0.021], time:102.646 ms, lr:0.01000
Epoch time: 109591.053 ms, per step time: 58.449 ms, avg loss: 0.021
Epoch:[  8/ 10], step:[ 1875/ 1875], loss:[0.069/0.019], time:103.502 ms, lr:0.01000
Epoch time: 108099.482 ms, per step time: 57.653 ms, avg loss: 0.019
Epoch:[  9/ 10], step:[ 1875/ 1875], loss:[0.070/0.016], time:111.135 ms, lr:0.01000
Epoch time: 110012.473 ms, per step time: 58.673 ms, avg loss: 0.016
```

通过在测试集测试，得到的准确率大约为 98.8%，截图如下：

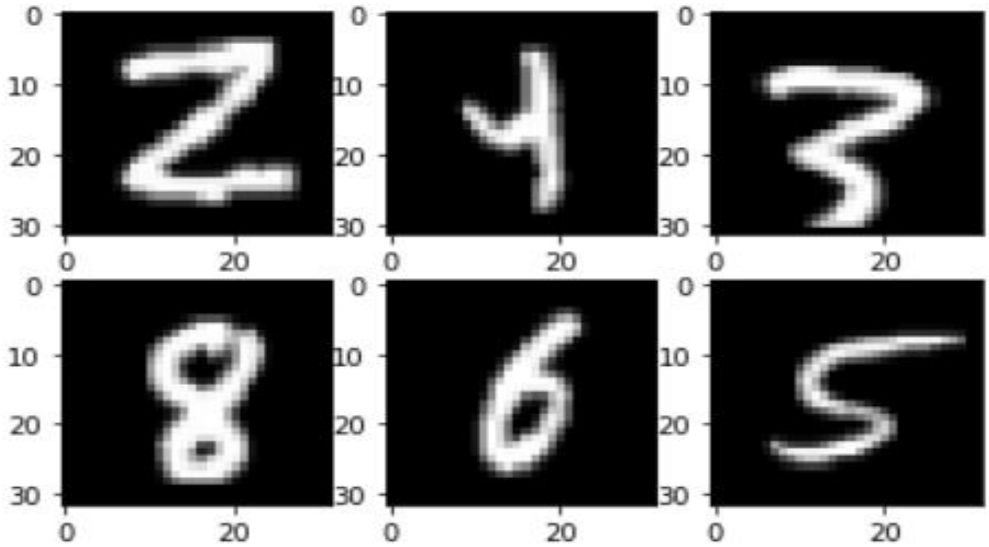
```
acc = model.eval(dataset_eval)

print("{}".format(acc))
```

```
{'accuracy': 0.9879807692307693}
```

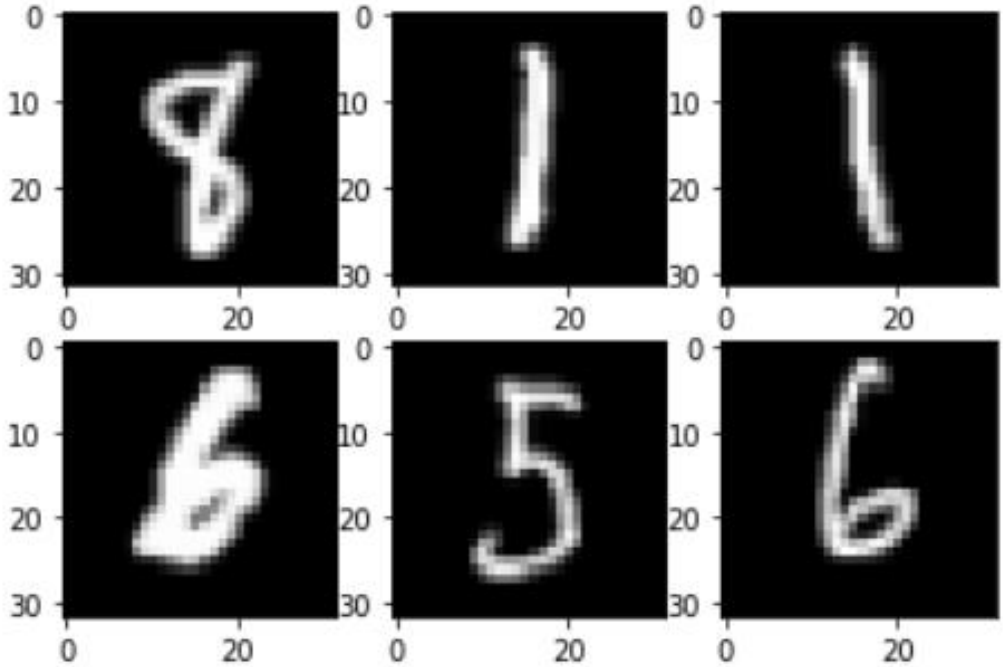
从测试集中随机抽取样本，已训练模型对预测分类结果和实际分类结果的可视化结果如下：（尝试了 3 次抽样预测，可以看到预测结果和实际结果几乎一致，更加直观地展示了模型的高准确率）



```
Predicted: "[9 4 9 6 7 6]", Actual: "[9 4 9 6 7 6]"
```



```
Predicted: "[2 4 7 8 6 5]", Actual: "[2 4 3 8 6 5]"
```



Predicted: "[8 1 1 6 5 6]", Actual: "[8 1 1 6 5 6]"

4.2 问题 2 垃圾分类

本问题需要实现对垃圾分类数据集上垃圾图片进行分类。模型微调过程中，一共修改了三次参数， 通过 args 文件指定运行参数，同时在 config 文件中结合当前数据集特点修改网络参数。模型一共训练了三次，第一次 batch_size 为 1、epoch_size 为 20，训练过程截图如下：

Epoch: [x / y] ：x 表示当前完成的 epoch 数，y 表示总共需要完成的 epoch 数。

per step time：表示每个 step 的平均处理时间。一个 step 通常指的是模型处理一个批次（batch）的数据所需的时间。

avg loss：是平均损失。

```
Complete the batch 2590/2593
Complete the batch 2591/2593
Complete the batch 2592/2593
Complete the batch 2593/2593
[WARNING] ME(39239:139892168599360,MainProcess):2025-05-19-16:16:37.992.301 [mindspore/nn/loss/los
s.py:161] '_Loss' is deprecated from version 1.3 and will be removed in a future version, use 'Los
sBase' instead.
epoch[1/20], iter[2593] cost: 14494.347, per step time: 5.590, avg loss: 103.170
epoch[2/20], iter[2593] cost: 6387.123, per step time: 2.463, avg loss: 76.571
epoch[3/20], iter[2593] cost: 5501.240, per step time: 2.122, avg loss: 70.897
epoch[4/20], iter[2593] cost: 5605.362, per step time: 2.162, avg loss: 62.816
epoch[5/20], iter[2593] cost: 5605.243, per step time: 2.162, avg loss: 59.694
epoch[6/20], iter[2593] cost: 5744.695, per step time: 2.215, avg loss: 57.502
epoch[7/20], iter[2593] cost: 5573.702, per step time: 2.150, avg loss: 54.171
epoch[8/20], iter[2593] cost: 5713.590, per step time: 2.203, avg loss: 54.099
epoch[9/20], iter[2593] cost: 5628.987, per step time: 2.171, avg loss: 51.848
```

```
epoch[10/20], iter[2593] cost: 5522.062, per step time: 2.130, avg loss: 50.603
epoch[11/20], iter[2593] cost: 5562.457, per step time: 2.145, avg loss: 46.984
epoch[12/20], iter[2593] cost: 5549.680, per step time: 2.140, avg loss: 48.087
epoch[13/20], iter[2593] cost: 5502.841, per step time: 2.122, avg loss: 48.824
epoch[14/20], iter[2593] cost: 5575.938, per step time: 2.150, avg loss: 47.699
epoch[15/20], iter[2593] cost: 5572.067, per step time: 2.149, avg loss: 45.115
epoch[16/20], iter[2593] cost: 5418.202, per step time: 2.090, avg loss: 45.295
epoch[17/20], iter[2593] cost: 5506.674, per step time: 2.124, avg loss: 45.611
epoch[18/20], iter[2593] cost: 5554.570, per step time: 2.142, avg loss: 45.392
epoch[19/20], iter[2593] cost: 5584.795, per step time: 2.154, avg loss: 45.563
epoch[20/20], iter[2593] cost: 5550.529, per step time: 2.141, avg loss: 42.493
total cost 749.8456 s
```

（图中的 warning 可忽略，因为这是由于 MindSpore 版本更新导致的函数可能弃用警告）

根据训练过程可以看出 loss 值比较大，意味着模型准确率可能不太高，验证准确率如下：

```
result:{'acc': 0.6884615384615385}
pretrain_ckpt=./ckpt_0/mobilenetv2_20.ckpt
```

Loss 值太大可能是由于 batch_size 太小 or 太大，或者 epoch_size 太小（当然还有其他原因，在此仅关注微调 batch_size 和 epoch_size 的值）。因此第二次训练 batch_size 改为 50，epoch_size 仍为 20，训练过程截图如下：

```
Complete the batch 43/51
Complete the batch 44/51
Complete the batch 45/51
Complete the batch 46/51
Complete the batch 47/51
Complete the batch 48/51
Complete the batch 49/51
Complete the batch 50/51
Complete the batch 51/51
[WARNING] ME(42537:140221757429568,MainProcess):2025-05-19-16:30:20.597.066 [mindspore/nn/loss/los
s.py:161] '_Loss' is deprecated from version 1.3 and will be removed in a future version, use 'Los
sBase' instead.
epoch[1/20], iter[51] cost: 1956.348, per step time: 38.360, avg loss: 2.358
epoch[2/20], iter[51] cost: 1064.800, per step time: 20.878, avg loss: 1.510
epoch[3/20], iter[51] cost: 1020.323, per step time: 20.006, avg loss: 1.254
epoch[4/20], iter[51] cost: 1102.323, per step time: 21.614, avg loss: 1.165
epoch[5/20], iter[51] cost: 1121.356, per step time: 21.987, avg loss: 1.110
epoch[6/20], iter[51] cost: 830.732, per step time: 16.289, avg loss: 1.031
epoch[7/20], iter[51] cost: 820.531, per step time: 16.089, avg loss: 1.014

epoch[8/20], iter[51] cost: 891.857, per step time: 17.487, avg loss: 0.984
epoch[9/20], iter[51] cost: 1122.813, per step time: 22.016, avg loss: 0.935
epoch[10/20], iter[51] cost: 1120.334, per step time: 21.967, avg loss: 0.927
epoch[11/20], iter[51] cost: 1127.183, per step time: 22.102, avg loss: 0.907
epoch[12/20], iter[51] cost: 1107.681, per step time: 21.719, avg loss: 0.912
epoch[13/20], iter[51] cost: 911.245, per step time: 17.868, avg loss: 0.868
epoch[14/20], iter[51] cost: 843.626, per step time: 16.542, avg loss: 0.842
epoch[15/20], iter[51] cost: 829.669, per step time: 16.268, avg loss: 0.823
epoch[16/20], iter[51] cost: 844.861, per step time: 16.566, avg loss: 0.835
epoch[17/20], iter[51] cost: 831.271, per step time: 16.299, avg loss: 0.816
epoch[18/20], iter[51] cost: 785.319, per step time: 15.398, avg loss: 0.809
epoch[19/20], iter[51] cost: 1108.641, per step time: 21.738, avg loss: 0.815
epoch[20/20], iter[51] cost: 1191.605, per step time: 23.365, avg loss: 0.794
total cost 204.0786 s
```

可以看到 loss 值有了显著的降低，预测此时模型准确率也会大幅提高，验证准确率结果如下：

```
result:{'acc': 0.888}
pretrain_ckpt=./ckpt_0/mobilenetv2_20.ckpt
```

但是觉得还不是特别高，因此又尝试着第三次修改 batch_size 为 60，epoch_size 为 25，训练结果截图如下：（batch_size 只要不是超过数据集大小即可，模型微调参数的过程就是不断尝试的过程嘛）

```
Complete the batch 42/43
Complete the batch 43/43
[WARNING] ME(43943:139788508772160,MainProcess):2025-05-19-16:37:18.984.422 [mindspore/nn/loss/los
s.py:161] '_Loss' is deprecated from version 1.3 and will be removed in a future version, use 'Los
sBase' instead.
epoch[1/25], iter[43] cost: 2000.668, per step time: 46.527, avg loss: 2.254
epoch[2/25], iter[43] cost: 753.127, per step time: 17.515, avg loss: 1.517
epoch[3/25], iter[43] cost: 790.534, per step time: 18.385, avg loss: 1.372
epoch[4/25], iter[43] cost: 752.280, per step time: 17.495, avg loss: 1.164
epoch[5/25], iter[43] cost: 768.482, per step time: 17.872, avg loss: 1.063
epoch[6/25], iter[43] cost: 765.839, per step time: 17.810, avg loss: 1.021
epoch[7/25], iter[43] cost: 747.392, per step time: 17.381, avg loss: 0.971
epoch[8/25], iter[43] cost: 763.511, per step time: 17.756, avg loss: 0.981
epoch[9/25], iter[43] cost: 735.279, per step time: 17.100, avg loss: 0.954
epoch[10/25], iter[43] cost: 772.546, per step time: 17.966, avg loss: 0.918
epoch[11/25], iter[43] cost: 742.867, per step time: 17.276, avg loss: 0.882
epoch[12/25], iter[43] cost: 766.045, per step time: 17.815, avg loss: 0.846
epoch[13/25], iter[43] cost: 744.219, per step time: 17.307, avg loss: 0.857
epoch[14/25], iter[43] cost: 781.977, per step time: 18.186, avg loss: 0.838
epoch[15/25], iter[43] cost: 752.012, per step time: 17.489, avg loss: 0.835
epoch[16/25], iter[43] cost: 716.527, per step time: 16.663, avg loss: 0.834
epoch[17/25], iter[43] cost: 782.201, per step time: 18.191, avg loss: 0.814
epoch[18/25], iter[43] cost: 745.710, per step time: 17.342, avg loss: 0.801
epoch[19/25], iter[43] cost: 817.078, per step time: 19.002, avg loss: 0.796
epoch[20/25], iter[43] cost: 726.872, per step time: 16.904, avg loss: 0.793
epoch[21/25], iter[43] cost: 773.746, per step time: 17.994, avg loss: 0.789
epoch[22/25], iter[43] cost: 764.591, per step time: 17.781, avg loss: 0.785
epoch[23/25], iter[43] cost: 730.000, per step time: 16.977, avg loss: 0.780
epoch[24/25], iter[43] cost: 750.248, per step time: 17.448, avg loss: 0.775
epoch[25/25], iter[43] cost: 777.274, per step time: 18.076, avg loss: 0.780
total cost 197.4708 s
```

相比第二次参数下的 loss 值又低了一些，验证模型准确率结果如下：

```
result:{'acc': 0.9125}
pretrain_ckpt=./ckpt_0/mobilenetv2_24.ckpt
```

（每次验证时验证训练过程中每个 epoch 结束后 loss 值最低的模型）

其实可以编写一些自动化调整参数的代码，在模型训练过程中动态调参，使得模型的准确率进一步提高，而非像实验中这样手动尝试着调参。但由于时间关系，本次实验并未实现该功能。但本次实验得到的模型准确率为 91%左右，也已经算是比较优秀的分类模型了。

5. 总结与分析

5.1 问题与解决方案

Question1： 问题 1 需要 MNIST 手写数字数据集，但目前 MNIST 数据集官网（http://yann.lecun.com/exdb/mnist/）已经无法下载，进入后只是显示如下图所示，没办法下载。

Solution1：从 gitee 上找到了分享的仓库和相关的教程（均已附在“参考文献”部分中）（此处我强烈建议在给下一届学弟学妹们出这道题时能附上这些资料链接，这些有用的链接也是我尝试了很多之后选出来的）

5.2 收获与思考

5.2.1 关于 MindSpore 的总结：

每每谈到主流的深度学习框架，我们总能想到 TensorFlow (Google) 和 PyTorch (Meta/Facebook)，此次课程设计让我了解到了华为推出的 MindSpore 这个深度学习框架。所以我首先去了解了一下关于华为这个 MindSpore 和我们常提的 TensorFlow、PyTorch 这两种到底有什么不同。阅读一些资料后感觉 MindSpore 一开始的定位就是原生支持华为自研的昇腾 AI 芯片（Ascend），而 TensorFlow/PyTorch 则主要支持 NVIDIA GPU 等，对国产芯片支持有限。因此 MindSpore 最大的特点可能就是在硬件支持上与其他两种深度学习框架不同，感觉 MindSpore 的研发初衷可能就是为了突破国外对我国芯片等硬件研发的限制吧。除此之外，MindSPore 既支持动态图，也支持静态图，是集成了 TensorFlow 和 PyTorch 的两种编程

模式，有利有弊吧，同时支持两种编程模式对开发的要求应该更高，而 MindSpore 从 2019 年才开源，其发展、追赶之路还有一段距离，目前生态支持也不是很完善。就结合我此次实验的过程来说，这个 MindSpore 的框架特别不好配，我在 anaconda 中创建了一个环境后专门配置 MindSpore，但一开始总是安装失败（我完全按照官网给出的教程和安装命令进行安装的），后来我直接在华为云上运行了，避免了配置环境的麻烦。但是我之前配置 PyTorch 时好像没有遇到什么难以解决的问题。可能是 MindSpore 的资料还不够多吧，遇到的很多问题都找不到提供解决方案的文档。

5.2.2 关于 Lenet5 的总结：

LeNet-5 是一个经典的深度卷积神经网络，其基本结构包括 7 层网络结构（不含输入层），其中包括 2 个卷积层、2 个降采样层（池化层）、2 个全连接层和输出层。MindSpore 的 MindSpore Vision 套件直接提供了 LeNet 网络模型接口 lenet， 只需要使用相关的函数，就可非常方便快捷地创建一个 Lenet5 网络。但我还是去了解了一下 Lenet5 的具体实现，将一个基于 Pytorch 实现的 Lenet5 模型用 MindSpore 重新写了一下，代码如下：

```
import mindspore
import mindspore.nn as nn
import mindspore.ops as ops
class LeNet5(nn.Cell):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1,
pad_mode='valid')
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1,
pad_mode='valid')
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(in_channels=16 * 4 * 4, out_channels=120)
        self.fc2 = nn.Dense(in_channels=120, out_channels=84)
        self.fc3 = nn.Dense(in_channels=84, out_channels=10)
        self.relu = ops.ReLU()
    def construct(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool1(x)
        x = self.conv2(x)
```

```
        x = self.relu(x)
        x = self.pool2(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x
```

5.2.3 关于 MobileNetV2 的总结：

MobileNetV2 是在 MobileNetV1 基础之上的改进。MobileNetV1 主要思想就是深度可分离卷积。上一个问题中提到的 Lenet5 就是最经典的普通卷积，因此我就去了解了一下什么是深度可分离卷积。

简单来说，深度可分离卷积相比于常规卷积可以减少参数，一定程度上提高速度。至于为什么可以达到这种作用，可能还需要我后续参考更多的资料和论文去理解，在此不再讨论。

而 V2 在 V1 基础上的创新就是线性瓶颈（Linear Bottleneck） 和倒残差结构 （Inverted Residuals），简单来说“线性瓶颈”就是经典的网络最后一层常用 ReLU 激活函数（非线性），但 MobileNetV2 最后只用线性（不加激活函数），信息可以保留得更多，还不容易丢失有用特征；“倒残差结构 ”就是普通残差结构（ResNet）是“宽→窄→宽”，即先减少通道数再加回去。而倒残差结构是“窄→宽→窄”，先把特征通道变多（升维），在高维空间做计算，然后再压缩回去（降维）。这两个特点跟我们课堂上所学的经典模型处理都是不一样的，但 MobileNetV2 的论文中却指出这样可以使得模型的准确率更高且文件更小。所以感觉给我最大的印象就是，一直以来的不一定是最好的，深度学习的模型也没有“最佳”，需要不断地尝试或者提出新的方法。

6. 参考文献（资料）

1.MNIST 手写数字识别在线运行网站（无需本地配置 MindSpore 环境，直接在 JupyterLab 界面运行）：

https://www.mindspore.cn/tutorials/zh-CN/r1.7/beginner/quick_start.html

2.MNIST 手写数字数据集可以通过该博客的百度网盘分享下载：

https://blog.csdn.net/KRISNAT/article/details/130670596

或者从以下仓库下载

https://gitcode.com/Resource-Bundle-Collection/d47b0?utm_source=highlight_word_gitcode&word=mnist%E6%95%B0%E6%8D%AE%E9%9B%86

3..垃圾分类教程（非常有用，只需将教程中的花卉数据集替换成垃圾分类数据集）

https://github.com/mindspore-courses/computer_vision/blob/master/fine_tune/fine_tune.md